Problem 1

Final Result

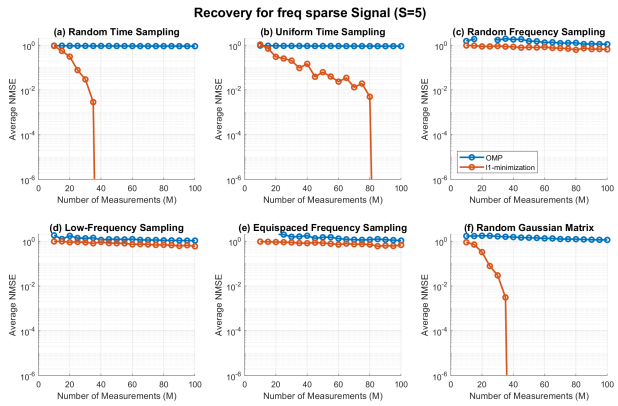

Figure 1: Frequency Sparse Signal Results

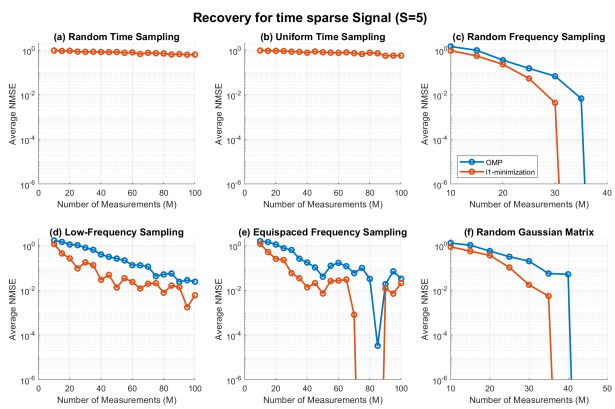

Figure 2: Time Sparse Signal Results

Matlab Code

```
clear; clc; close all;

%% Setup Parameters
N = 256;          % Signal length
S = 5;            % Sparsity level
M_vals = 10:5:100; % Measurements to test
num_trials = 50; % Number of trials for averaging

% Different scenarios for the experiment
signal_types = {'time_sparse', 'freq_sparse'};
matrix_types = {'a', 'b', 'c', 'd', 'e', 'f'};
algo_names = {'OMP', 'l1-minimization'};

%% Results Storage
avg_nmse = zeros(length(signal_types), length(matrix_types), length(M_vals), length(algo_names));

%% Simulation Loop
for signal_idx = 1:length(signal_types)
    signal_type = signal_types{signal_idx};

    for matrix_idx = 1:length(matrix_types)
        matrix_type = matrix_types{matrix_idx};

        for m_idx = 1:length(M_vals)
            M = M_vals(m_idx);

            trial_nmse = zeros(num_trials, length(algo_names));

            parfor trial = 1:num_trials
                % 1. Generate Signal
                [x, ~] = generate_signal(N, S, signal_type);

                % 2. Generate Sensing Matrix
                A = generate_matrix(M, N, matrix_type);

                % 3. Acquire Measurements
                y = A * x;

                % 4. Recover Signal using different algorithms

                % OMP Recovery
                x_omp = SOLVE_OMP(A, y, S);
                nmse_omp = norm(x - x_omp)^2 / norm(x)^2;

                % l1-minimization Recovery
                x_l1 = recover_l1(A, y, N, signal_type);
                nmse_l1 = norm(x - x_l1)^2 / norm(x)^2;

                trial_nmse(trial, :) = [nmse_omp, nmse_l1];
            end

            % Average over trials
            avg_nmse(signal_idx, matrix_idx, m_idx, :) = mean(trial_nmse, 1);
        end
    end
end


%% Plot Results
matrix_titles = {
    '(a) Random Time Sampling', ...
    '(b) Uniform Time Sampling', ...
    '(c) Random Frequency Sampling', ...
    '(d) Low-Frequency Sampling', ...
    '(e) Equispaced Frequency Sampling', ...
    '(f) Random Gaussian Matrix'
    };

for signal_idx = 1:length(signal_types)
    figure('Name', ['Signal: ' signal_types{signal_idx}], 'NumberTitle', 'off', 'Position', [100, 100, 1200,
            700]);
    sgtitle(['Recovery for ' strrep(signal_types{signal_idx}, '_', ' ') ' Signal (S=5)'], ...
        'FontSize', 16, 'FontWeight', 'bold');

    for matrix_idx = 1:length(matrix_types)
        subplot(2, 3, matrix_idx);
        hold on;
        for algo_idx = 1:length(algo_names)
            plot(M_vals, squeeze(avg_nmse(signal_idx, matrix_idx, :, algo_idx)), ...
                'o-', 'LineWidth', 2, 'MarkerSize', 6);
        end
        hold off;

        title(matrix_titles{matrix_idx}, 'FontSize', 12);
        xlabel('Number of Measurements (M)');
        ylabel('Average NMSE');
        grid on;
        set(gca, 'YScale', 'log');
```

```
        ylim([1e-6, 2]);

        if matrix_idx == 3
            legend(algo_names, 'Location', 'southwest');
        end
    end
end

%% Helper Functions
function [x, alpha] = generate_signal(N, S, signal_type)
arguments
    N
    S
    signal_type {mustBeMember(signal_type, ['time_sparse', 'freq_sparse'])}
end

alpha = zeros(N, 1);
q = randperm(N);
alpha(q(1:S)) = randn(S, 1);

switch signal_type
    case 'time_sparse'
        x = alpha;
    case 'freq_sparse'
        x = idct(alpha);
    otherwise
        error('Unknown signal type specified.');
end
end

function A = generate_matrix(M, N, signal_type)
arguments
    M
    N
    signal_type {mustBeMember(signal_type, ['a', 'b', 'c', 'd', 'e', 'f'])}
end
% Generates an M x N sensing matrix based on the specified type.

I = eye(N);
F = dct(eye(N)); % Orthonormal DCT matrix

switch signal_type
    case 'a' % Random sampling in the time domain
        q = randperm(N);
        A = I(q(1:M), :);

    case 'b' % Uniform subsampling in the time domain
        indices = round(linspace(1, N, M));
        A = I(indices, :);

    case 'c' % Random sampling in the frequency domain
        q = randperm(N);
        A = F(q(1:M), :);

    case 'd' % Low-frequency sampling
        A = F(1:M, :);

    case 'e' % Equispaced frequency sampling
        indices = round(linspace(1, N, M));
        A = F(indices, :);

    case 'f' % Sampling in a random domain (Gaussian)
        A = randn(M, N);
        A = orth(A')'; % Orthonormalize the rows

    otherwise
        error('Unknown matrix type specified.');
end
end

function x_hat = recover_l1(A, y, N, signal_type)
arguments
    A
    y
    N
    signal_type {mustBeMember(signal_type, ['time_sparse', 'freq_sparse'])}
end
switch signal_type
    case  time_sparse
        M_matrix = A;
    case  freq_sparse
        F = dct(eye(N));
        M_matrix = A * F';
    otherwise
        error('Unknown signal type specified.');
end

[m_rows, ~] = size(M_matrix);

f = [zeros(N, 1); ones(N, 1)];
```

```
A_eq = [M_matrix, zeros(m_rows, N)];
b_eq = y;


A_ineq = [eye(N), -eye(N); -eye(N), -eye(N)];
b_ineq = zeros(2 * N, 1);

lb = [-inf(N, 1); zeros(N, 1)];


options = optimoptions('linprog', 'Display', 'none');


v = linprog(f, A_ineq, b_ineq, A_eq, b_eq, lb, [], options);

if isempty(v)
    warning('linprog did not find a solution. Returning a zero vector.');
    x_hat = zeros(N, 1);
    return;
end

z_hat = v(1:N);

switch signal_type
    case  time_sparse
        x_hat = z_hat;
    case  freq_sparse
        x_hat = idct(z_hat);
    otherwise
        error('Unknown signal type specified.');
end
end

function [x_hat, error] = SOLVE_OMP(A, y, S, tol)
% Orthogonal Matching Pursuit (OMP) algorithm to recover a sparse signal.
%
% Inputs:
%   A     - Sensing matrix (M x N)
%   y     - Measurement vector (M x C)
%   S     - Sparsity level (number of non-zero elements in x)
%   tol   - Optional tolerance to stop iterating
%
% Output:
%   x_hat - Reconstructed sparse signal (N x C)
%   error - Error between A*x_hat and y
arguments
    A   {mustBeNumeric}
    y   {mustBeNumeric, mustHaveSameRows(A,y)}
    S   {mustBeInteger, mustBePositive}
    tol {mustBeReal, mustBePositive} = 1e-15
end

% Get dimension of sensing and measures
[~, N] = size(A);
[~, C] = size(y);

r = y; % Initialize the residual
T = double.empty(0,S); % Initialize the support set
x_hat_T = zeros(S,C);  % Initialize the sparse signal estimate

for i = 1:S

    % Add most correlated column indice to support set
    [~, J] = max(abs(A'*r));
    T_i = union(T,J);

    % Solve the lq to get best estimate with the support set
    x_hat_T_i = pinv(A(:,T_i))*y;

    % Get residual for best estimate
    r_i = y - A(:,T_i)*x_hat_T_i;

    if  (norm(r_i,2) >= norm(r,2)) || (norm(r, 2) < tol)
        break;
    else
        % Update residual, support set, and sparse x then continue
        r = r_i;
        T = T_i;
        x_hat_T = x_hat_T_i;
    end
end

% Build full signal estimate vector
x_hat = zeros(N,C);
if ~isempty(T)
    x_hat(T,:) = x_hat_T;
end
% Calculate Measurement error for full estimate
error  = norm(y-A*x_hat, 2);
end
```

```
function mustHaveSameRows(a,b)
% Validate that a and b have the same number of rows
if ~isequal(size(a,1), size(b,1))
    error('Inputs must be have the same number of rows.');
end
end
```